

AgentWorkShop: A Reusable Agent-Team Supervisory Framework for Multi-Scenario Industrial Optimization

Anonymous Authors

Abstract—Industrial optimization applications are difficult to reuse because task orchestration, field-protocol integration, production context, execution authority, and intervention evidence are commonly implemented as application-specific logic. This paper presents AgentWorkShop, a reusable agent-team supervisory framework that connects deployed team identities to semantic data-acquisition and write-control nodes through explicit capabilities. The framework resolves line, product, recipe, and run context at execution time and records admission, approval, actuation, readback, process observation, submitted judgment, and compensation as distinguishable evidence states. Five protocol families, self-describing node contracts, versioned production objects, plugin driver interfaces, and a typed SDK provide common integration surfaces; scenario-specific models, objectives, guards, and control policies remain external inputs. Evaluation uses real protocol stacks connected to author-implemented dynamic simulators, with deterministic external policies invoking the governed tools; no language-model decision quality or optimization-algorithm superiority is claimed. In an integrated reference run, 83 of 83 checks pass: configured out-of-window probes are rejected on the writable lines; single-node and nine-device AgentTeam missions both close their governed loops; and all four scripted process missions attain their objective predicates. Task completion, device readback, and process attainment remain separately recorded, so their agreement in this run is evidence rather than a collapsed status. The results support an inspectable, simulation-tier architecture for reusable industrial supervisory execution, not a certified safety or autonomous-control function.

Index Terms—Agent teams, industrial informatics, recipe optimization, supervisory control, write governance.

I. INTRODUCTION

INDUSTRIAL optimization often occurs above the regulatory control layer. Engineers adjust recipes, schedules, and setpoints to meet quality, throughput, energy, or discharge objectives while PLCs and local controllers retain fast regulation. The supervisory decision is only one part of the intervention. A usable execution path must also establish which deployed actor may change which parameter, which limits are active for the current product and run, whether the device accepted the command, and whether the process subsequently improved.

These concerns recur across plants whose physics and objectives differ: injection molding couples part weight to pressure, time, and temperature; wastewater treatment (WWTP) balances effluent constraints against aeration and dosing cost; annealing trades material properties against line speed; and BOPET (biaxially oriented polyester) production coordinates multiple stretching sections. All of them need an auditable

bridge from a team-level task to heterogeneous acquisition and control nodes.

Large language model (LLM) agents are being studied for industrial diagnosis, engineering assistance, and PLC-program generation [1–4], including simulated controller and machine interaction [5, 6]. Yet orchestration frameworks and protocol connectivity do not by themselves answer who is authorized, which production limits apply, or how a write is linked to later evidence (Section II). Moreover, a task state such as COMPLETED can coexist with a missed process objective.

AGENTWORKSHOP addresses this execution gap through member–node bindings and shared line, product, recipe, and run context. Nodes expose engineering quantities rather than register addresses; bindings scope sanctioned tools; and intervention records retain the proposal, execution result, observation, and submitted judgment. Fig. 1 contrasts this organization with application-specific integration; fast regulation, certified protection, and the choice of optimization algorithm remain outside the framework.

The paper makes four contributions:

- 1) a **capability-based industrial execution architecture** that connects agent-team identity, semantic data-acquisition (DAQ) and write-control (DCW) nodes, heterogeneous protocol drivers, and line–product–recipe–run context while keeping task membership distinct from physical-write authority;
- 2) a **reusable integration boundary** consisting of self-describing node and driver contracts, production objects, plugin extension points, and a typed SDK. The evaluated reference adapter projects scenario blueprints into these common services, while process models and policies remain external;
- 3) an **evidence-preserving governed intervention workflow** that separates admission, optional approval, actuation, readback, process observation, submitted judgment, compensation, and policy-driven backstop actions; and
- 4) a **layered simulation-tier evaluation** spanning five protocol stacks, a mechanism ablation, a deterministic controller, two AgentTeam missions (one node and a nine-device line), and four process scenarios, with task state reported separately from process-objective attainment.

The experiments use real software protocol stacks against author-implemented dynamic process simulators under deterministic scripted policies.

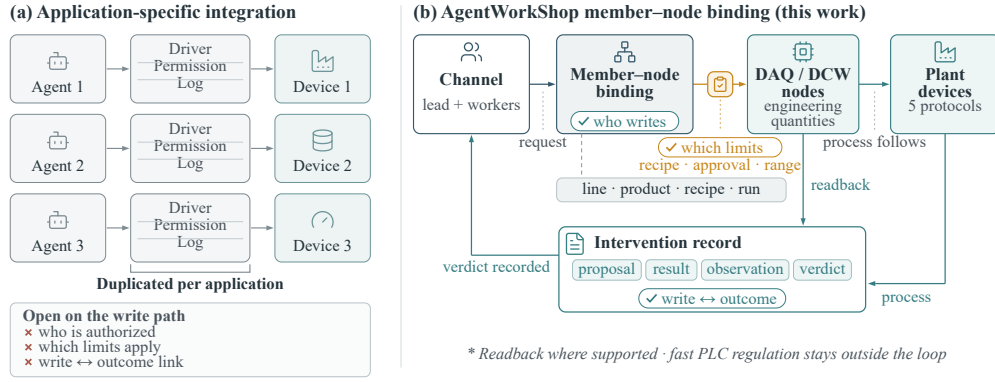


Fig. 1. Conceptual supervisory integration: (a) application-specific links, duplicating drivers, permissions, and logging per application; (b) governed member-node binding in AGENTWORKSHOP: requests pass recipe, approval, and range checks under shared production context, and intervention records return readback, observation, and verdict. Integration patterns are compared, not optimization performance; PLC regulation remains outside the agent-team loop.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based manufacturing has long connected organizational roles to industrial capabilities [7]. Recent LLM-oriented work spans industrial knowledge graphs [8], diagnosis [9], PLC generation and verification [3, 4], digital-twin configuration [10], OPC UA machine control [6], and simulated fault handling [5, 11]; InstructMPC combines language-derived objectives with model-predictive control [12]. These studies motivate agent-facing industrial tools; the shared execution layer that carries authority and evidence across scenarios, our focus here, is complementary rather than a new controller or reasoning method.

B. Orchestration, Industrial Models, and Governance

AutoGen, LangGraph, and CrewAI coordinate tasks and tools [13–15]; the Model Context Protocol (MCP) and Agent-to-Agent (A2A) specifications standardize tool and inter-agent interfaces [16, 17]. Such frameworks coordinate reasoning and tool invocation, but the exposed tool is usually a function signature with no line, product, or recipe context and no record that separates admission from execution; industrial deployment additionally needs engineering semantics, driver behavior, and production context. ISA-88, ISA-95, ISO 23247, and IEC 63278 provide complementary models for recipes, operations, and digital twins [18–21]: AGENTWORKSHOP’s versioned recipes and run dispatch instantiate a reduced subset of this stack—master-recipe-style versions over a line-product-run resource model—without batch procedure models or full equipment hierarchy. IIoT and shop-floor agent architectures similarly emphasize contextualized integration [22]; the nearest neighbors [23, 24] target control decoupling and task decomposition, while AGENTWORKSHOP ties each write to production-scoped authority and a stored evidence chain.

Permissions, range checks, approvals, and audit trails have DCS/SCADA/MES and IEC 62443 counterparts [25]. Formal shielding, control-barrier functions, and simplex run-time assurance offer stronger specified properties [26–28]. In contrast, AGENTWORKSHOP implements source-dependent checks and a rule-based backstop but makes no formal protection guarantee; the backstop evaluated in Section V-C is an alarm-style

rule in the spirit of ISA-18.2 and EEMUA-191 [29, 30], not a run-time assurance bound. Formal methods aside, human oversight itself requires more than an approval dialog [31, 32].

C. Evaluation Scope

AgentBench and consequential tool-use benchmarks assess agent behavior and policy compliance [33–35]; TEP and SWaT support process-monitoring research [36, 37]. Indirect prompt injection and its defenses motivate compromised-tool studies [38, 39]. Our evaluation instead separates source checks, protocol execution, mechanism ablation, controller trials, and scripted multi-scenario missions; adversarial prompting and comparisons with other agent frameworks are outside scope.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP links team coordination to plant interaction through industrial nodes and production context (Fig. 2). Channels coordinate deployed members and tasks; controller services mediate node operations; production objects identify the line, product, recipe, and run. This organization is a deliberate separation of concerns: a team plane decides what to investigate and propose, an integration plane presents devices as semantic nodes, a governance plane decides whether a specific request is admissible, an operations plane supplies production context and evidence, and an extensibility plane adds drivers and services through sanctioned registration points. Table I summarizes the resulting functional surface and, for each plane, what remains scenario-specific. The decomposition is what makes the framework reusable: scenario knowledge is concentrated in blueprints, signal maps, objective predicates, and driver configuration rather than spread through the team scheduler, approval service, or persistence code.

A. Supervisory Core and Nodes

The Node.js core hosts channel routing, industrial services, and protocol adapters. A channel contains a lead, workers, tasks, messages, and retrievable memory; WebSocket, MCP, A2A, REST, CLI, and TUI entry points let a team be deployed by a UI, an external harness, or a programmatic client. Engine

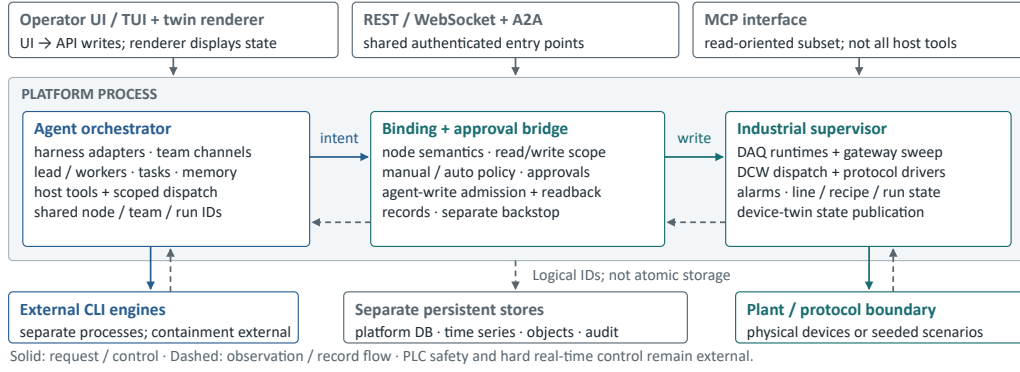


Fig. 2. System architecture and execution boundaries. The platform hosts team routing, binding and approval, industrial supervision, and records. Operator interfaces sit above it; agent engines, data stores, protocol endpoints, and the plant remain separate. Shared identifiers provide attribution but do not make storage and actuation transactional.

TABLE I

FUNCTIONAL SURFACE OF AGENTWORKSHOP AND ITS REUSE BOUNDARY. EACH ROW IS IMPLEMENTED AS A PLATFORM SERVICE, NODE ABSTRACTION, OR EXTENSION CONTRACT; SCENARIO INTEGRATION CHANGES THE RIGHTMOST COLUMN WHILE PRESERVING THE COMMON EXECUTION PATH.

Plane	Core functions	Evidence or control boundary	Scenario-specific input
Team coordination	Channels, lead/worker roles, state-tracked tasks, messages, retrievable memory, multi-engine adapters, WebSocket/MCP/A2A/REST/CLI/TUI entry points	Member identity and task state are distinct from physical-write authority	Team topology, goal, stopping rule
Industrial integration	DAQ/DCW nodes, engineering units, calibration, precision, connection tests, read/write drivers, rotating acquisition runtime, digital-twin and live-stream hooks	Protocol acceptance, readback, freshness, connection and transport errors	Device blueprint, signal map, driver config
Governance	Member-node bindings, line permissions, hard and contextual windows, approval mode, post-write hold, source-specific routes, compensation and backstop	Admission, approval, actuation, readback, observation, judgment, compensation, and roll-back are separately recorded	Limits, recipe windows, approval policy
Production operations	Line-product-recipe-run hierarchy, versioned recipes, run dispatch, alarms, sampled histories, optimization records, audit and operator surfaces	Production context is attached to requests and records; PLC protection remains independent	Production objects, objective and guards
Extensibility	Hot-reloadable plugins, server/browser hooks, new DAQ drivers, agent tools, settings groups, i18n panels, typed SDK, schema-driven runtime configuration	Extension code uses sanctioned services instead of arbitrary plant I/O	Optional plugin, external controller, or deployment adapter

adapters preserve member and node identity across runtimes, and channel membership alone confers no write authority.

DAQ nodes convert device readings to engineering quantities; DCW nodes translate engineering setpoints into driver commands and, where supported, readback. Per-node runtimes manage timing and in-flight work, and gateway services supply connection state, active recipe context, and write bookkeeping. The built-in semantic surface covers Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, and HTTP/REST. The deployment connects to author-implemented dynamic process simulators through the same node and driver contracts a plant deployment would use; the simulators are deterministic per seed but are not validated digital twins. This is the first reuse boundary: a scenario declares signals, units, scaling, precision, and driver configuration, and the team loop remains unchanged. Fast regulation and independent protection stay in plant controllers.

Driver contracts are deliberately narrow. A driver declares how a semantic quantity maps to its transport payload, whether the endpoint is writable, and whether a readback is available;

the controller, not the driver, resolves member authority and active limits. Connection, transport, and protocol errors are stored with the request instead of being folded into one success flag, so a timeout, a rejected frame, and a written-but-unconfirmed value remain distinguishable. Register addresses, data types, and byte order appear only here, keeping engineering quantities free of vendor-specific encoding at higher layers, so adding a fieldbus family leaves the team loop untouched.

Acquisition is deliberately supervisory rather than hard real-time. Each DAQ node runs its own cadence with due-time scheduling and in-flight tracking under a rotating sweep, so a slow endpoint delays only its own samples. Every reading carries a timestamp, a quality state, and a freshness age, and the response window later used by a verdict is a query over that history rather than a blocking read. This lets a proposal be judged against the process response rather than the command echo—and is why a bucket average can lag the device by one cadence.

B. Bindings, Context, and Reuse Boundaries

A member–node binding grants access to a sanctioned tool on a specific node and selects automatic or approval-mediated execution. Operator line permissions (none, read-only, and operate) authorize binding creation and mutation; runtime invocation is then authorized by the persisted member–node capability, not channel membership. Hard, baseline, product, and recipe intervals are resolved and re-checked at every request rather than cached: revoking a binding or switching the active recipe takes effect on the next request, without re-deploying the team. The same request path serves an operator console, an external harness, and an agent engine, so authority does not depend on the client; only the evidence record differs, carrying the acting member and the proposal’s origin.

Production objects are versioned rather than overwritten. A recipe is a versioned named definition, so a run is dispatched against a known-good version, reverted non-destructively, and later compared with the values actually applied; a governed intervention is read against the definition that was active when it was admitted, not against the configuration current at inspection time. The extension surface follows the same rule: a plugin may register server hooks, routes, agent tools, DAQ drivers, and browser panels, but only through sanctioned services rather than plant I/O of its own, so an added driver inherits admission, readback, and evidence capture.

The second reuse boundary is exercised by a reference onboarding adapter built on the platform APIs: it reads a scenario blueprint, performs an idempotent “ensure” over the registered simulator endpoints, exports driver configurations, creates one semantic node per signal, and attaches a recipe; existing lines are reused by scenario label without rebuilding the team. The SDK exposes the same REST and typing surface to ordinary programs, and runtime settings are schema-described and hot-reloadable, so deployment policies and sampling cadences change without editing the mission loop.

The stores share identifiers rather than a transaction domain: platform state uses SQLite and files; measurement and object backends use TimescaleDB and MinIO with local fallbacks. A device may therefore accept a write even if later bookkeeping fails, and records lack a run identifier, so time-range queries can cross run boundaries unless callers constrain them—boundaries disclosed rather than hidden.

For a BOPET thickness task, a worker queries the thickness node and proposes changes to casting speed, fast-roll speed, or rail width through bound DCW nodes; the same mission pattern serves injection molding, WWTP, and annealing with only blueprint, guards, and policy differing. Task completion remains distinct from device acknowledgment and target attainment.

C. Reference Scenario-Onboarding Contract

The reference integration follows a bounded onboarding contract rather than duplicating the supervisory application: the engineer declares a blueprint of device and signal identifiers, engineering units, writable/read-only roles, initial setpoints, and plant-model parameters. The platform performs a differential “ensure” pass—creating missing devices, repairing

drifted signals, verifying ports and connectivity, and loading the scenario model without disturbing other registered lines—projects the blueprint into semantic DAQ/DCW nodes and a recipe, and tags the line so a later run can reuse it idempotently. A channel is then instantiated from the common team template with bindings to the required nodes, and a mission supplies the objective, guards, response windows, and policy; every observation, write, and judgment flows through the same governed services. The adapter demonstrates reuse of platform services, not configuration-only commissioning of arbitrary physical plants. The ensure pass is differential and idempotent, so a scenario can be re-applied after a partial failure, and the projection is non-destructive because each scenario is tagged and its nodes resolved through that tag.

D. Governed Optimization Task Lifecycle

A governed optimization task is a life cycle rather than a single tool call, and AGENTWORKSHOP exposes each transition separately. The task enters a channel carrying goal, line, product, recipe, run, and task identifiers; a lead member claims it and dispatches the work to a worker holding member–node bindings on the relevant nodes. The worker then repeats one fixed job: read the controlled quantity over a bounded time window, propose a single setpoint inside the interval resolved for that request, invoke the governed write, wait for the process response, and record a verdict. Section V-B shows this path in operation.

Every iteration leaves more than a written value. The time-range read returns samples, freshness, and the active window; the write opens an optimization record holding the prior and requested values, the admitting interval, the acting member, and timestamps; readback, where supported, closes the command path. The parameter journal attributes the change to its source; attainment is evaluated against the scenario predicate, and the task closes with a workflow terminal state (Section IV gives the admission, readback, judgment, and compensation semantics).

This separation bounds what “automatic optimization” means here. The framework automates the execution of the loop—reading process state, admitting and applying candidate setpoints, waiting for a response, recording evidence, and accepting or compensating the intervention. It does not prescribe a search algorithm, choose the stopping rule, or claim that a language model autonomously selects the best action. A controller, a scripted policy, a conventional optimizer, or an agent engine may occupy the strategy slot, while authority, evidence, and closure remain platform responsibilities. A mission can therefore reach its workflow terminal state while missing the process objective, and the two are never merged into one status.

IV. GOVERNED EXECUTION MECHANISMS

A supervisory intervention must preserve distinctions that are often collapsed in an agent tool call: permission to propose, admission of a value, driver acceptance, readback, process response, a caller’s verdict, and compensation. AGENTWORKSHOP exposes these as separate, distinguishable evidence states (Figs. 3 and 4).

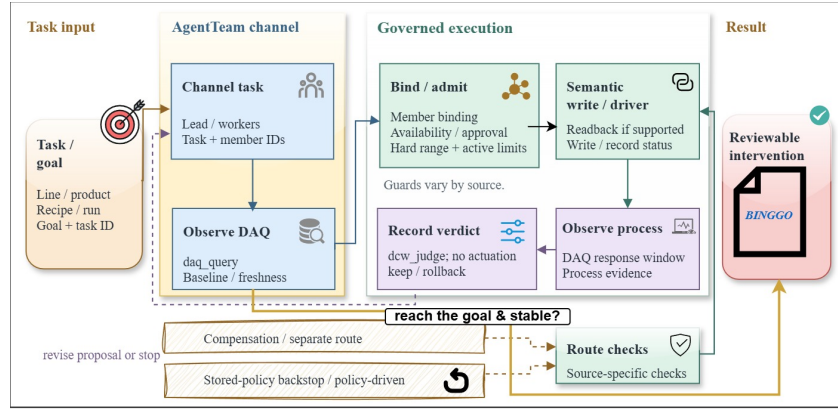


Fig. 3. Implemented supervisory workflow. A task carrying line, product, recipe, run, goal, and task identifiers enters an AgentTeam channel, which assigns lead/worker member identifiers and queries DAQ for a baseline and freshness. Governed execution then (i) resolves the member binding and admits the request against availability or approval, the hard range, and active limits; (ii) performs the semantic driver write and readback when supported; (iii) observes a DAQ response window; and (iv) records a non-actuating `dcw_judge` verdict. If the goal is not reached and stable, the strategy may revise or stop. Compensation follows a separate route, and a stored-policy backstop may invoke source-specific route checks. The output is a reviewable intervention, not a guarantee of process success.

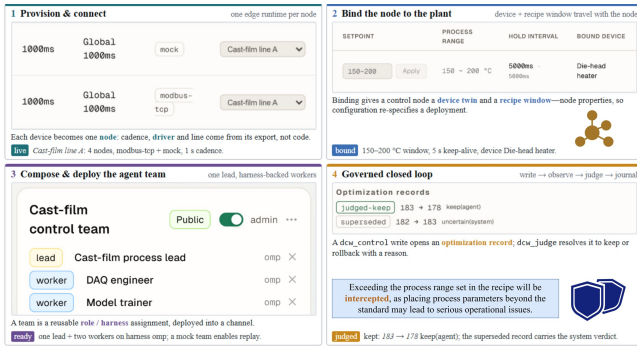


Fig. 4. The implemented path on the running operator surfaces: (1) a device export becomes a semantic node with cadence, range, and driver; (2) a binding attaches the node to a line, a device, and a recipe window; (3) a channel deploys one lead and two workers; (4) a governed closed loop opens an optimization record and stores a verdict. Running UI/API surfaces, not benchmark outcomes.

A. Task, Observation, and Admission

The strategy receives a task and goal in production context and decides what to propose and when to stop. The platform governs execution; decision variables are semantic node parameters—pressure, dosing rate, line speed, rail width—in engineering units. Authority belongs to deployed member-node bindings, not the channel, prompt, or task template.

Before a proposal, `daq_query` obtains a baseline or response window and reports freshness. For an ordinary agent setpoint request on node n , the implemented admission interval is

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \quad (1)$$

where $\mathcal{H}_n$ is the hard engineering range and the remaining terms are applicable baseline, active-product, and active-recipe limits. Missing optional limits add no restriction. This intersection constrains submitted values; it is not a learned feasibility region or a physical safety certificate. Availability, binding state, and approval policy are checked separately.

B. Semantic Write and Source-Specific Routes

A semantic card exposes quantity, unit, precision, range, current value, freshness, and recipe context. The controller resolves the member binding and active limits, then the driver serializes the command; readback is attempted only when supported and closes the command path, not the process-response path.

After a successful agent or manual write, the node installs a configurable post-write hold window, `writeLockSeconds`, whose default is 30s: a further agent/manual request for the same node during that window receives HTTP 429 `WRITE_FREQUENT`. The lock is installed only after the driver reports success; recipe-dispatch and rollback writes are exempt because they use separate operational routes, zero disables it, and `AW_BENCH_MODE=1` bypasses it for benchmark harnesses. The hold is per node and applies only to agent-origin requests: it guards a supervisory loop whose response window may be comparable to the sampling cadence, and it is not a rate limit on the plant.

Guards vary by source, as shown in Fig. 3: ordinary agent writes use the full binding and admission path; recipe dispatch supplies run context and applies its route-specific checks; hold heartbeats share runtime write exclusion but establish no member authority; line start installs active context before dispatching recipe parameters individually, and stop clears that context and closes retained open records. These multi-step operations are not atomic with device state.

For a request u_n , the framework stores the attempted actor and context, executes the driver call, and stores write/readback status, so a driver error, missing readback, or bookkeeping failure remains visible rather than being recast as process success. Agent-origin ownership and cooldown checks reduce some conflicting updates but are not universal guards for manual, recipe, or system routes.

C. Observation, Judgment, and Compensation

An accepted write opens an optimization record containing the prior and requested values, applicable limits, actor,

and timestamps. The strategy then queries process observations over a response window. `dcw_judge` records `keep`, `rollback`, or `uncertain` plus a reason; it performs no actuation and does not independently validate the evidence, so a scripted `keep` can coexist with a missed objective. The verdict is stored with the record it closes and with the response window that was actually queried, so a later reader can see what evidence was in front of the caller at the time, and a verdict is never rewritten in place: a compensation or a backstop action adds a new record instead of editing the previous one.

Compensation is a new, separately admitted write to a stored baseline or anchor-derived value; it may fail after the original command succeeded, and a multi-node sequence may only partially complete. Recipe rollback restores a definition, not physical PLC values. These separations keep command status, record status, and process recovery independently inspectable. Fig. 4 shows the resulting observe–write–judge path end to end, from device export through binding, channel deployment, and the recorded verdict.

A stored-policy backstop periodically evaluates open records: in the implemented default it waits at least 120 s, checks no more often than every 30 s, and triggers after at least three returned points or bucket averages are outside the stored window; automatic recovery is attempted only while fewer than two retained records for the node are already marked rolled back. Asynchronous queries, retained-history counting, and later record changes mean this cadence is neither a deadline nor a formal run-time assurance bound.

D. Human Approval Boundary

Manual bindings insert a timed approval wait before an agent setpoint or physical compensation request; automatic bindings omit the wait but retain applicable checks. The pending request shows member, node, requested value, request-time interval, and expiry. Approval reserves neither the interval nor the device state, so binding and admission are checked again before execution. Expiry and refusal are recorded separately, and duplicate pending requests are suppressed.

The current decision endpoint authenticates a user but does not enforce a line-scoped approver role, and reads, verdict recording, and some configuration operations do not use physical-write approval. Thus this mechanism demonstrates workflow mediation, not segregation of duties or certified human supervision.

E. Design Rationale and Reuse Boundary

The strategy engine is replaceable rather than trusted as a control primitive; authorization, readback, task termination, and process attainment remain distinct. The stable plane retains bindings, semantic I/O, evidence, approval, compensation, and backstop handling, while a scenario supplies its blueprint, limits, objective, response window, and policy. This confines scenario-specific work without claiming physical portability.

Two rules keep that boundary stable. First, scenario code may declare and configure but may not execute plant I/O: blueprints, signal maps, guards, and policies are data and

strategy, while every write still passes through admission, driver dispatch, and evidence capture. Second, the platform never infers a process objective; attainment is computed by the scenario predicate, so the same records can be re-evaluated against a revised predicate without rewriting history.

V. EXPERIMENTAL EVALUATION

The evaluation asks whether one governed supervisory path can be exercised, inspected, and repeated across heterogeneous industrial jobs rather than whether one optimizer dominates another. Four questions organize the evidence: protocol closure and governance attribution (Q1); the AgentTeam optimization workflow on one node and on a multi-node line (Q2); the deterministic cast-film loop (Q3); and objective attainment for four process missions (Q4). The path they instantiate is the one shown in Fig. 4.

A. Testbed, Evidence Snapshot, and Scope

The snapshot is integrated run 20260921184204-101c (seed 42, commit e692df1, harness 9f2756dce4a4cbb3, completed September 21, 2026 (19:08 UTC)); raw reports and mission logs are archived under that run identifier, and the ablation companion is 20260921181234-e11ite. The simulators behind the five stacks are deterministic per seed but not validated digital twins. Deterministic scripted policies, not language models, call the governed tools under a deployed member identity; two live agent-engine turns serve as a slot-occupancy check only, and the four-scenario campaign runs one trajectory per scenario at the fixed seed-42 initial states. Physical-plant transfer, safety certification, and optimizer ranking remain out of scope.

B. Test Procedure and System-Level Profile

The procedure is one 20-phase integrated suite scoring 83/83 checks with no warnings or failures, an author-defined regression summary. Each phase probes one claim: preset loading against an offline optimum $J^* = 89.894$; provisioning, DAQ, governed writes, and out-of-window interception on five stacks; the observe–write–judge cycle; the AgentTeam missions; records, rollback, approval, audit, and alarms; recipe lifecycle and the parameter layer; the controller benchmark; the second preset; the backstop drill; and the four-scenario campaign.

Fig. 5 reports two system-level metrics. Panel (a) is each phase's wall time on a log scale—the end-to-end clock of all twenty phases, from preset provisioning through the four-scenario campaign—and it answers where the run's 26 min actually goes. Three tiers appear. Deterministic governance steps—admission, binding checks, record writes, recipe lifecycle—complete in milliseconds to seconds: the platform's own cost per intervention is validation logic, not optimization work. The long tail belongs to the replaceable strategy slot and to stored observation cadences: the two live agent-engine turns take 156.3 s and 637.3 s, and the backstop drill spans 130.2 s at the stored evaluation cadence. This is the intended cost shape: the framework stays thin, and the expensive tiers are the ones a

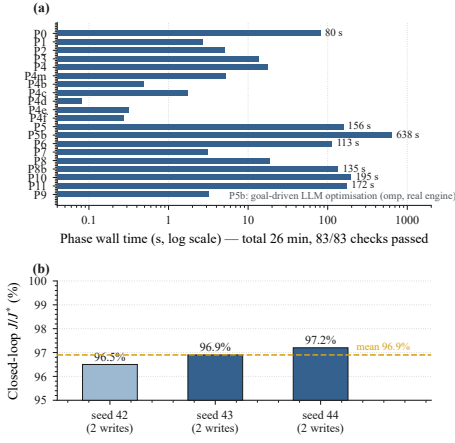


Fig. 5. Run 20260921184204-101cat system level. (a) What each phase of the suite costs in clock time (log scale): provisioning, governance drills, missions, and the campaign; deterministic governance phases are short, while the two live agent-engine turns and the cadence-bound backstop drill dominate. (b) Closed-loop objective J after two governed writes as a fraction of the offline optimum J^* , per seed; no write is rejected, and the oracle gap is expected. Panel (a) is environment telemetry, not a service-level bound.

deployment would swap or re-time. Panel (b) is the governed closed-loop objective J after two writes, normalized by the simulator's offline optimum J^* ; it measures how closely a permission-checked, journaled loop follows an oracle policy that sees the model. Three seeds land between 0.965 and 0.972 with no write rejected, previewing Q3.

C. Q1: Protocol Closure and Governance Attribution

On the five protocol lines the suite stores 55 DAQ samples through real drivers, including a Modbus RTU-over-TCP satellite. Governed writes are accepted and read back on Modbus TCP, OPC UA, MQTT, and HTTP with per-line medians of 47.8 ms, 19.8 ms, 19.3 ms, and 32.9 ms; the median over all governed writes pooled across the four lines is 29.954 ms. The identical admission, record, and verdict code path serves every request regardless of stack, so the residual spread between the fastest and slowest line is attributable to the transports rather than to the governance logic. Each of the four writable lines rejects 6/6 configured out-of-window probes while accepting its legal write, so all 24 probes are intercepted and the interlock discriminates between violating and admissible requests rather than merely blocking.

Two drills then separate the truth paths that a single success flag would collapse. A frozen-measurement drill accepts a write and a consistent readback while the measurement stays out of window, and the backstop records a system-judged rollback 130.2 s later: readback closes the command path, the backstop closes the record, and neither is a settling time. A second preset with different signals and dynamics is commissioned by configuration alone: three writable lines and two satellite acquisition nodes are created with 0 framework code changes, five nodes sample, and the writable lines reject 9/9 probes. Counting both presets, 33 configured probes are rejected across seven writable lines (four in the first preset, three in the second), so the reuse claim is exercised rather than asserted: the same admission code produced the same discrimination on signals it had never seen.

TABLE II
GOVERNANCE ABLATION ON COMPANION RUN
20260921181234-ELITE (THREE DETERMINISTIC REPETITIONS WITH IDENTICAL COUNTS; 18 PROBES PER ARM: 12 HARD-RANGE, 6 WINDOW-CLASS).

Arm	Rejected	Executed breaches	False blocks
Full governance	18/18	0	0
No interlock	12/18	6	0
No readback check	18/18	0	0
Both relaxed	12/18	6	0

The parameter layer passes 4/4 and makes the interval geometry of Section IV-A observable: a semantic surface without register leakage, and a four-layer chain (node [50, 200], parameter [143.7, 156.3], product [146.22, 153.78], recipe [141, 159]) narrowing to one admitted interval, with out-of-layer requests intercepted. Each layer contributes a bound, and a rejection is attributable to the layer that excludes the request. A manual binding instead suspends a request visibly and unblocks it on approval, after which the write takes effect at the device (automated-roundtrip latency 461 ms): workflow mediation, not physics.

Table II attributes the rejection rather than assuming it, on a pipeline-tier companion fixture kept separate from the five protocol lines; counts are identical across the three deterministic repetitions. Full governance and the no-readback arm reject 18/18 probes, whereas removing the interlock, alone or with the readback check, lets the six window-class probes execute and be journaled. Two readings follow. First, prevention is the interlock's job: the readback check contributes nothing to interception because it closes the command path after execution, yet it is retained because it is what makes a written-but-unconfirmed command visible. Second, attribution survives removal: the six executed breaches still appear in the journal, so a relaxed configuration degrades from prevention to detection instead of failing silently. The hard range is structurally unby-passable and the soft window is its removable part; no arm records a false block.

D. Q2: The AgentTeam Optimization Workflow and Its Results

The workflow ran end to end on the cast-film line and then on a larger BOPET line (Section III-D). On the single-node mission one of three permitted writes sufficed: the setpoint moved from 200 to 204.704, the record was opened and judged, the observation settled at 204.70 against 204.704 ± 0.75 , the journal attributed the change to the agent identity, and the task closed COMPLETED. The mission demonstrates targeting discipline as much as closure: the strategy stopped after one write inside a budget of three because the first response already satisfied the predicate.

The BOPET mission is the multi-node case: nine devices across five protocol families, 49 signals, 30 writable and 19 readable, all created by the *ensure* pass and projected with driver tests passing on all nine endpoints. Thickness started at $28.10 \mu\text{m}$ against $25.0 \pm 0.7 \mu\text{m}$. Three governed writes on three distinct nodes—casting speed $32 \rightarrow 33.8$, fast-roll speed $118 \rightarrow 120.3$, and rail width $3000 \rightarrow 3035$ —brought it to $25.66 \mu\text{m}$, each opening a record, receiving

TABLE III

FOUR SCENARIO MISSIONS IN RUN 20260921184204-10LC. WALL TIME IS THE MISSION CLOCK; THE BOPET CLOCK WAS NOT SEPARATELY RETAINED.

Mission	Controlled variable (window)	Start	Final
Injection	Part weight (g), 32.5 ± 0.35	31.25	32.17
WWTP/A2O	Aerobic DO (mg/L), 3.4 ± 0.6	0.21	3.50
Annealing	Hardness (HV), 95 ± 6	134.52	100.08
BOPET	Thickness (μm), 25.0 ± 0.7	28.10	25.66

keep, and leaving an agent-attributed journal entry, while melt temperature stayed near 291°C inside $268\text{--}300^\circ\text{C}$ and the parent task reached COMPLETED. In these two missions, scaling from one node to nine did not scale writes with node count—one write on the single-node line, three on the nine-device line—consistent with per-write rather than per-node admission, record, and verdict overhead.

The strategy slot is also usable by a live engine: a bounded agent turn reached COMPLETED in 156.3 s, and a goal-driven turn took 637.3 s to drive thickness to $51.7\ \mu\text{m}$ against a self-selected target of $52.0 \pm 0.8\ \mu\text{m}$ across three writes and three verdicts, then saved the recipe. The two turns bracket the autonomy range the slot tolerates: the difference lies in deliberation and in the extra writes the goal-driven turn chooses, not in the evidence chain, which both share. This is slot occupancy through the common chain, not model quality.

E. Q3: The Deterministic Closed Loop

Seeds 42–44 each issue two governed writes with no rejection, raising the cast-film objective J (larger is better) from $J_0 \approx 67.2$ to J/J^* of 0.965, 0.969, and 0.972 against the simulator’s offline reference $J^* = 89.894$. The residual gap to unity is expected: J^* is an oracle computed with full model knowledge, which an online policy that sees only process observations cannot in general match, and the loop spends its writes where the response is largest. What the benchmark establishes is narrower: the governed path passes a conventional controller’s writes through unmodified—admissible, journaled, read back—with no ungoverned baseline run, so the overlay’s transparency is shown for admission and evidence handling, not measured as a performance delta.

F. Q4: Four Process Missions

The campaign runs four lines concurrently, each ensured from a blueprint, projected to nodes, and given one channel (Table III); all four attain their archived predicates, evaluated from subsequent observations rather than from the task state. Fig. 6 archives how: in every panel the curve is the observed controlled variable, the hatched band its objective window, triangles the governed writes, and the filled terminal marker the archived observation that satisfied the predicate. Each mission iterates the same loop—read the window, write inside the admitted interval, observe the response, judge—so the four trajectories are four executions of one governed loop, not four bespoke pipelines.

Panel (a): injection molding starts at 31.25 g with low hold pressure; two writes raise hold pressure from 45 to 58.2 and hold time from 6 to 10.64 s, finishing at 32.17 g inside $32.5 \pm$

$0.35\ \text{g}$ in 26.48 s with the flash and sink guards inside their limits. It is the minimal case: the response to the first write is nearly monotone, so the policy spends its budget and stops.

Panel (b): the wastewater mission is longest because it has two stages: aeration, caustic, and coagulant dose first bring dissolved oxygen from 0.21 into $3.4 \pm 0.6\ \text{mg/L}$ and satisfy the effluent guards on COD, ammonia, total phosphorus, pH, and turbidity; the policy then lowers blower frequency and caustic dose, reverting two blower steps that lose compliance margin and keeping two caustic reductions. Its 15 writes end at $3.50\ \text{mg/L}$ in 169.27 s and cut the first compliant operating-cost index from 181.5 to 179.1—a simulator-internal figure, not a utility bill. The two reverted blower steps are the mission’s most informative events: the writes were admissible and journaled, but the post-write observation lost compliance margin and the verdict stage caught it: rejection of a bad proposal lives in the evidence, not in the admission gate.

Panel (c): annealing starts far outside the hardness window (134.52 HV at 140 m/min): raising zone-3 temperature to 850°C brings hardness to 99.46 HV, one zone-2 trim (710 to 722°C) and two line-speed steps (140 to 152, then 164 m/min) hold $95 \pm 6\ \text{HV}$ while raising speed 17.1%, and an attempt at 176 m/min is reverted for leaving the margin. The guard here is multi-objective—hardness and throughput must hold together—and the predicate “inside the window and at least 5% more throughput” holds in 102.76 s only because the margin-violating speed was undone; the reversion record is what makes the final predicate checkable. Panel (d): BOPET is the larger line again: its campaign entry is the nine-device mission of Section V-D executed under the campaign harness (the same optimization records serve both reports), moving thickness from 28.10 to $25.66\ \mu\text{m}$ inside $25.0 \pm 0.7\ \mu\text{m}$; completion and the predicate agree there, yet they are still stored separately, so a later missed objective would stay visible.

G. Cross-Scenario Reading

Read together, the four missions bound what the framework contributes. The platform-side behavior is invariant—admission, driver I/O, records, verdicts, closure—while the write economy varies by a factor of more than seven (2 for injection, 15 for wastewater, 5 for annealing, 3 for BOPET) and the mission clock by a factor of more than six (26.48 s to 169.27 s over the three retained clocks; the BOPET clock was not retained). That variation is produced by construction by the scenario-specific slot: blueprint, guards, and policy. The three reversions (two blower steps in WWTP, one speed step in annealing) show the verdict layer doing real work; that all four terminal markers in Fig. 6 lie inside their bands is an observed outcome of this run, not a structural property—COMPLETED and predicate attainment remain separate records. Margins are narrow but on record: three terminals sit within 1% of a band edge (0.02 g, $0.04\ \mu\text{m}$, 0.92 HV to the nearest bound), and the compliant cost figure improves 1.3%. The evidence is simulation-tier, supporting reusable supervisory execution and repeatability rather than field optimality or physical transfer.

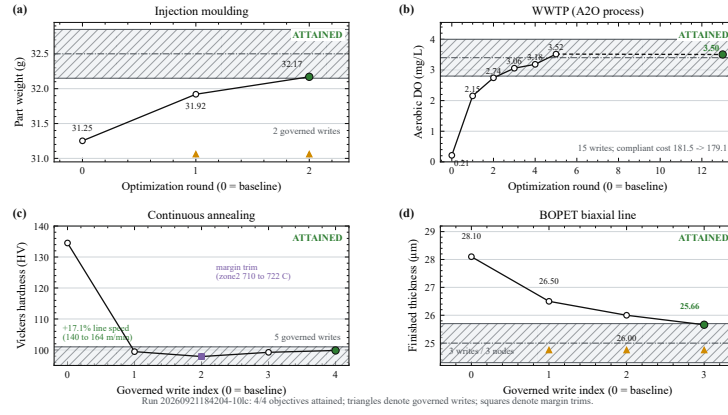


Fig. 6. Archived trajectories of the four missions in run 20260921184204-101c. Hatched bands are the objective windows and the dash-dotted line is the target. Filled terminal markers are archived observations satisfying the predicate; triangles mark governed writes and the square in (c) marks the zone-2 margin trim. All four terminal markers lie inside their objective window. Curves are simulator observations under scripted policies, not physical measurements.

VI. DISCUSSION AND LIMITATIONS

A. Industrial-Informatics Interpretation

The principal result is that one supervisory path carried four different jobs while keeping command closure, submitted judgment, task state, and process attainment as separate records. In run 20260921184204-101c all four scenario predicates hold and the governed closed loop reaches J/J^* between 0.965 and 0.972 (mean 0.969). That agreement does not collapse the records: a *keep* is still the caller's verdict, a completed task is still a workflow state, and a process predicate is still computed from subsequent observations.

The separation supports diagnosis: a binding or range rejection calls for a permission change, a driver or readback failure for execution investigation, and an accepted write followed by an adverse response for revision or compensation. The two reverted blower steps in the wastewater mission show that the verdict, not the write count, identifies a rejected proposal.

B. Validity and Reproducibility

The snapshot fixes the run identifier, seed, commit, harness fingerprint, and raw artifacts. Initial state, asynchronous sampling, response windows, and simulator timing still affect deterministic policies; the controller optimum and the scenario predicates are internal oracles; and the ablation has three deterministic repetitions and therefore supports mechanism attribution, not statistical inference; and latencies are environment observations, not service-level bounds. Evidence remains simulation-tier, from author-implemented dynamic simulators rather than validated digital twins. No physical-device campaign, operator study, TEP/SWaT replay, or external-framework comparison is reported; the four process models broaden structural variety without showing transfer to actual equipment; and records lack run identifiers, so poorly bounded queries may mix production runs.

C. Deployment Boundaries

Execution and recovery are non-atomic: bookkeeping may fail after actuation, the post-write hold is a per-node anti-frequency guard rather than a stability proof, readback agreement need not imply process recovery, and multi-node com-

pensation may partially succeed; within-range oscillation, storage failure, clock error, and worst-case recovery time remain unevaluated.

Bindings govern sanctioned tools, not arbitrary I/O from a privileged engine, so deployment requires least privilege, endpoint security, and IT/OT separation [25, 40]. Independent interlocks and any required IEC 61511 protection must remain: AGENTWORKSHOP is neither a safety instrumented function nor a substitute for plant protection [41]. Approval can also introduce fatigue, automation bias, and over-reliance [31, 32, 42, 43], and the present endpoint lacks line-scoped approver authorization; deployment would require fail-closed decisions, segregation of duties, and alarm-path failure tests [30].

VII. CONCLUSION

AGENTWORKSHOP contributes a reusable supervisory execution architecture rather than a new process optimizer: a stable path linking agent-team identities, semantic industrial nodes, production context, governed writes, readback, observations, verdicts, and compensation, exercised by a 20-phase suite, two AgentTeam missions, a three-seed closed loop, and four process missions. In this run five protocol stacks execute through the common node abstraction, all configured full-governance probes are rejected, and all four mission predicates hold, while the records keep command acceptance, readback, submitted judgment, and task completion distinct from those predicates. Claims of rapid commissioning, optimization superiority, autonomous LLM control, physical-plant transfer, and safety assurance require additional evidence.

DATA AVAILABILITY AND DISCLOSURE

Artifacts are archived under `bench/results/20260921184204-101c` (four-arm ablation: 20260921181234-ellipse); reproduction also needs the simulator source and reset state, and the snapshot lacks an immutable simulator-tree fingerprint, which the archival package must include. AI assistants supported editing and evidence checking under author direction; the authors remain responsible for all claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Sci. Rep.*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Comput. Ind.*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakihi, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 979–991, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IGK: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Trans. Ind. Inform.*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, pp. 1–4, *arXiv preprint arXiv:2405.18092*.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, *arXiv preprint arXiv:2505.02076*.
- [12] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179.
- [13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [14] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024, accessed: Sep. 20, 2026.
- [15] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIinc/crewAI>, 2024, accessed: Sep. 20, 2026.
- [16] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024, accessed: Sep. 20, 2026.
- [17] Linux Foundation, "Agent2Agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025, accessed: Sep. 20, 2026.
- [18] International Society of Automation, "ISA-88.00.01-2010—batch control—part 1: Models and terminology," Standard, 2010.
- [19] —, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.
- [20] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [21] International Electrotechnical Commission, "IEC 63278-1:2023: Asset administration shell for industrial applications—part 1: Asset administration shell structure," International Standard, 2023.
- [22] A. Gautam, M. R. Aryal, S. Deshpande, S. Padalkar, M. Nikolaenko, M. Tang, and S. Anand, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, pp. 511–523, 2025.
- [23] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.
- [24] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.
- [25] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.
- [26] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [27] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [28] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.
- [29] International Society of Automation, "ANSI/ISA-18.2: Management of alarm systems for the process industries (iec 62682)," Standard, 2016.
- [30] Engineering Equipment and Materials Users Association, "EEMUA 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.
- [31] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.
- [32] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [33] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, *arXiv preprint arXiv:2308.03688*.
- [34] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [35] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. Maben, R. Mehta, W. Chi, L. Jang, Y. Xie, S. Zhou, and G. Neubig, "Theagentcompany: Benchmarking LLM agents on consequential real world tasks," *arXiv preprint arXiv:2412.14161*, 2024.
- [36] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.
- [37] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [38] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [39] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupe>
- [40] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [41] International Electrotechnical Commission, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.
- [42] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [43] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.